

Programmieren (T3INF1004)

DHBW Stuttgart

Christian Bader

christian.Bader@lehre.dhbw-stuttgart.de

Christian Alexander Holz

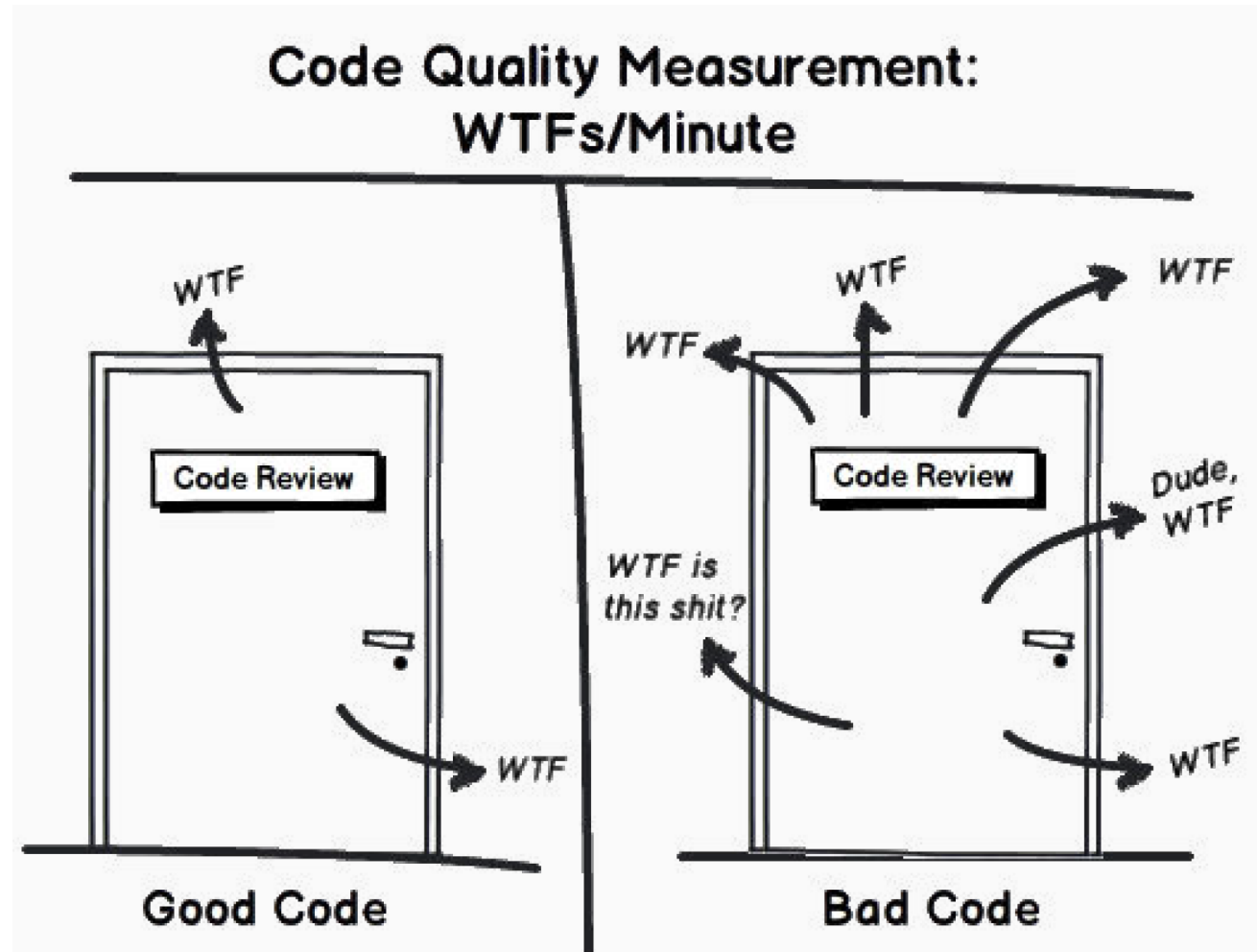
christian.holz@lehre.dhbw-stuttgart.de



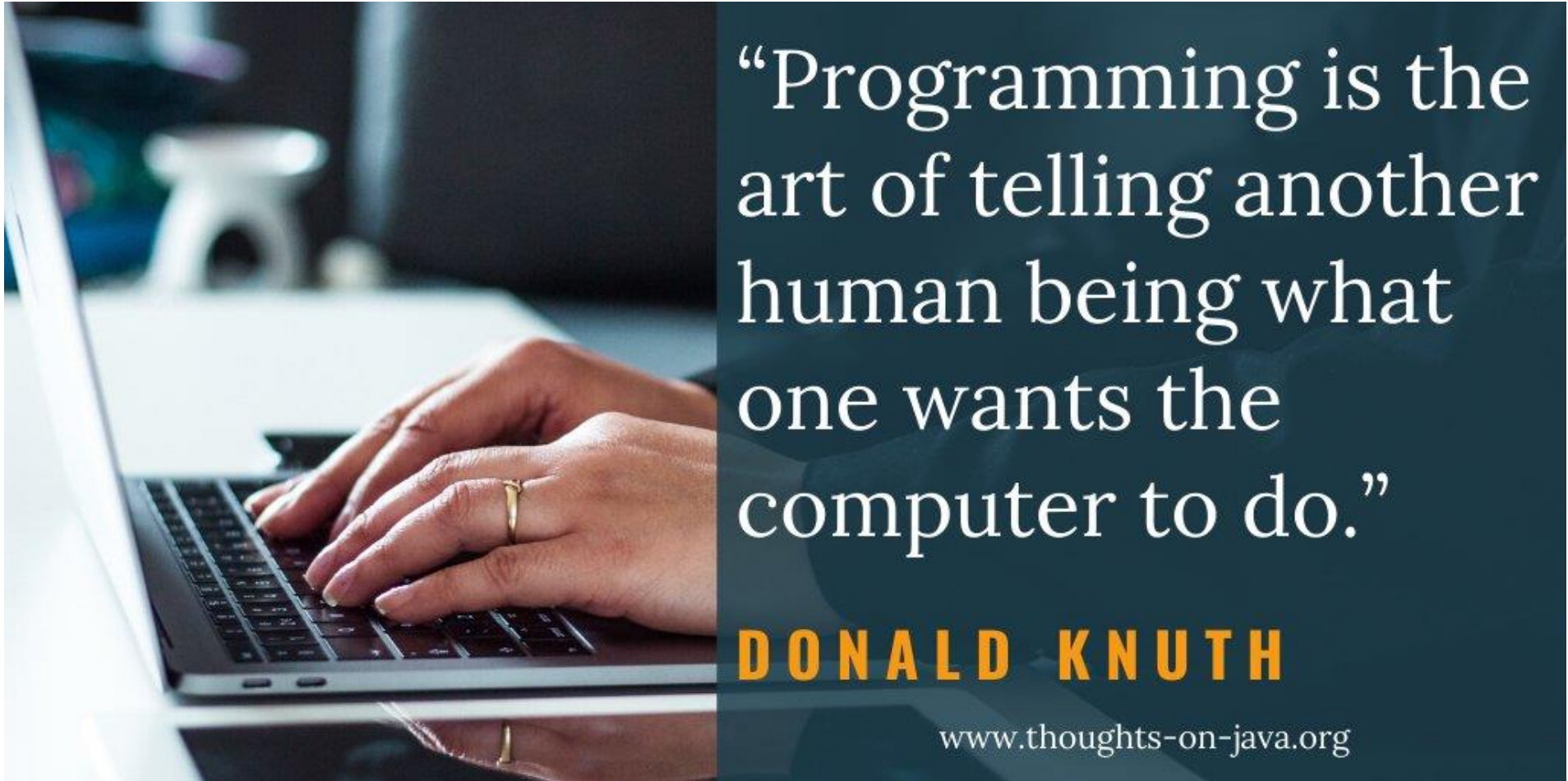
Kapitelübersicht

- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++
- 4 Die Standard Template Library (STL)
- 5 Clean Code
- 6 Test-driven Development

Die einzig echte Messgröße für guten Code...



Schlechte Code Qualität fährt Projekte gegen die Wand!



Kapitelübersicht

- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++
- 4 Die Standard Template Library (STL)
- 5 Clean Code
- 6 Test-driven Development

Kapitel 1: Einführung

1. „Hello C++ World!“ und Set-up
2. Wiederholung C

Was ist C++?

- 1979 von Bjarne Stroustrup entwickelt (Dänemark)
- **Stärke von C++:**
 - Effizientes, schnelles maschinennahes Programmieren
 - Optimiert für **Laufzeitperformance**
 - Mächtige Sprache mit vielen Möglichkeiten
- **Schwäche von C++:**
 - Nicht optimiert für **Entwicklungs-Performance**
 - Nicht optimiert für **Compile-time-Performance**
 - „Mit viel Macht kommt viel Verantwortung“: **Komplex** und **fehleranfällig** und damit schwerer wartbar (z.B. im Vergleich zu Python, Java, ...)
- Einsatz besonders auch im Embedded Bereich z.B. in Automobilindustrie:
Feld-Performance wichtiger als Entwicklungs- oder Compile-Time-Performance



Hello World!

Live

Integrated Development Environment (IDE)

Visual Studio Code

- In der Vorlesung wird VS-Code auf Windows benutzt (eigentlich kein IDE)
- Sie können natürlich einen andere IDE nutzen, ich kann nur im Zweifel nicht helfen
- Mächtiger Code Editor für nahezu alle Programmiersprachen
- *Light Weight* und schnell, kein eigener Compiler wie z.B. bei Visual Studio
- Erweiterungen können über **Pakete** installiert werden (es gibt alles mögliche an hilfreichen Erweiterungen)
- Runterladen von *Extensions* über *Marketplace*



Mein persönlicher Lieblings Codeeditor

Set-up Visual Studio Code



- Download z.B. [hier](#)
- Zum Arbeiten mit C++ folgen Sie dieser [Anleitung](#)
- Compiler: z.B. MinGW-x64 wie in Anleitung

Example: Install MinGW-x64

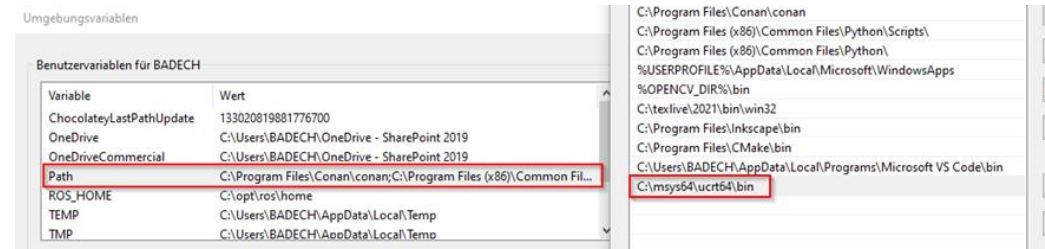
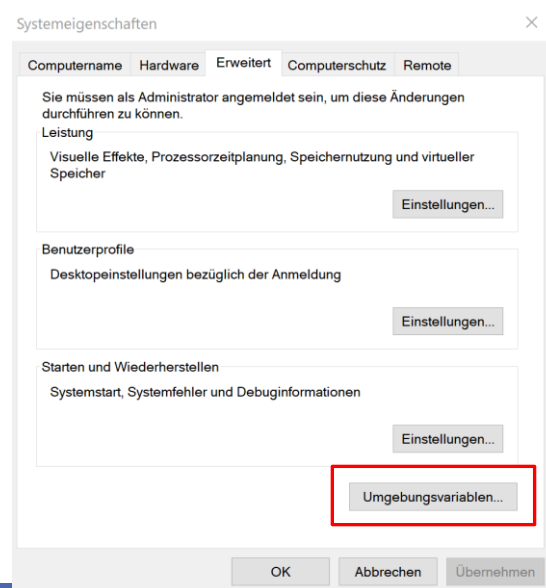
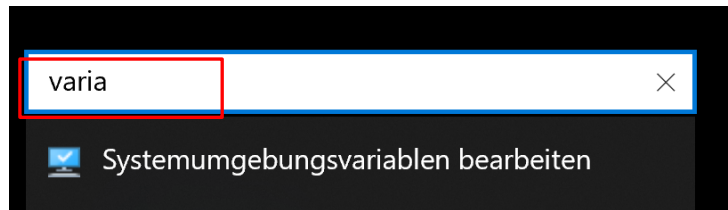
We will install Mingw-w64 via [MSYS2](#), which provides up-to-date native builds of GCC, Mingw-w64, and other helpful C++ tools and libraries. You can download the latest installer from the MSYS2 page or use this [link to the installer](#).

Follow the **Installation** instructions on the [MSYS2 website](#) to install Mingw-w64. Take care to run each required Start menu and `pacman` command, especially Step 7, when you will install the actual Mingw-w64 toolset (`pacman -S --needed base-devel mingw-w64-x86_64-toolchain`).

Set-up Visual Studio Code (Windows)



- Hinzufügen von `..\mingw64\bin` zum Pfad über Windows Suchleiste



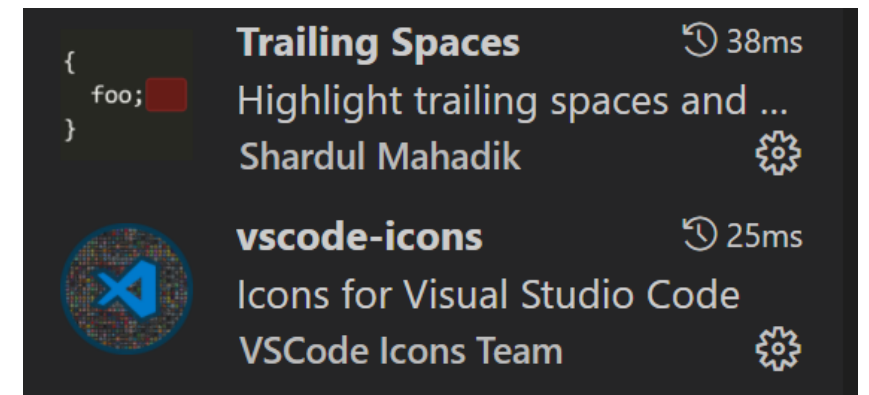
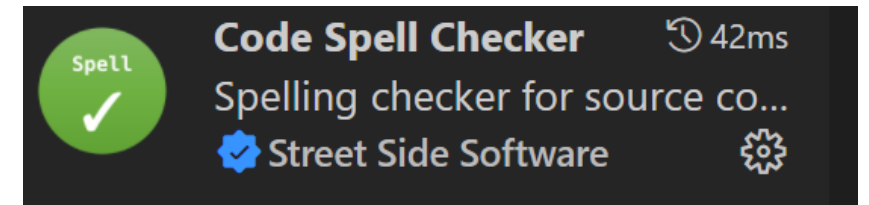
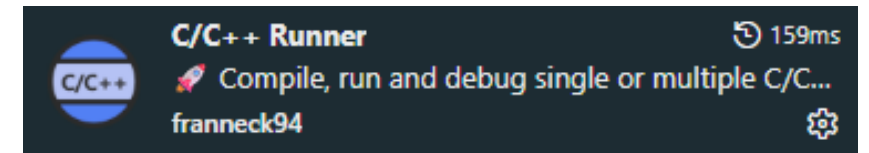
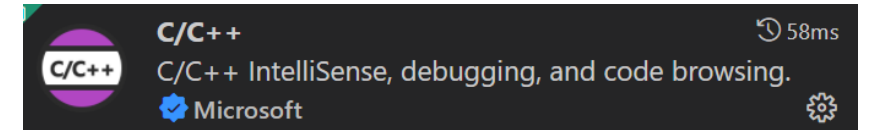
- Pfad Variable anklicken
- Manuell den Dateipfad von `mingw64\bin` einfügen (dort wo Sie es hin installiert haben, hier ist der Default Pfad angegeben)

```
MINGW64:/c/Users/druep
druep@DESKTOP-6635JKO MINGW64 ~
$ g++ --version
g++.exe (Rev10, Built by MSYS2 project) 11.2.0
Copyright (c) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

- In einer Konsole (z.B. VSCode oder Git Bash) überprüfen ob `g++` installiert ist

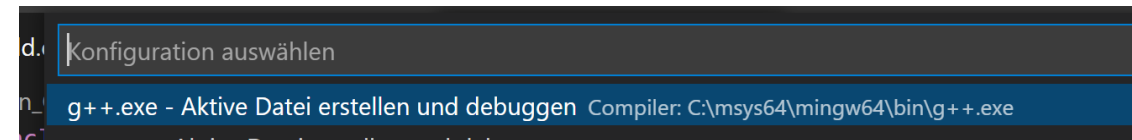
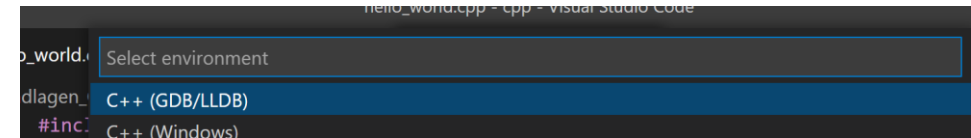
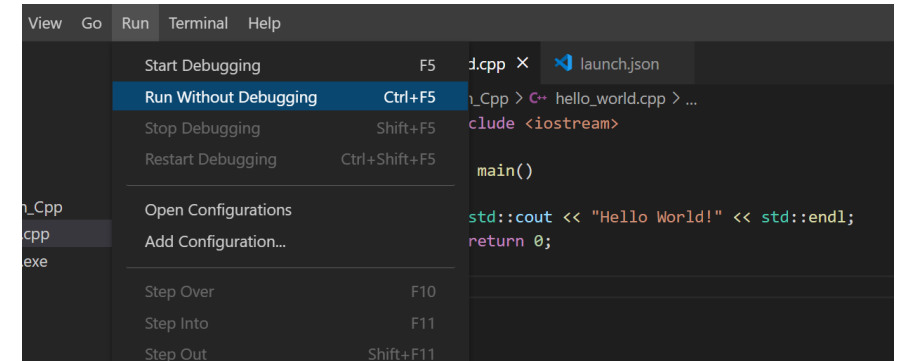
Weitere VS Code Extensions

- VS Code Extensions (Beispiele):
 - C/C++ IntelliSense: Highlighting und Support
 - Schnelles Kompilieren und Ausführen von C++ Files (nicht nötig wenn Debugger genutzt wird)
 - Code Spell Checker
 - Trailing Spaces
 - vscode-icons
 - Git Graph
 - C/C++ Runner
 - ...



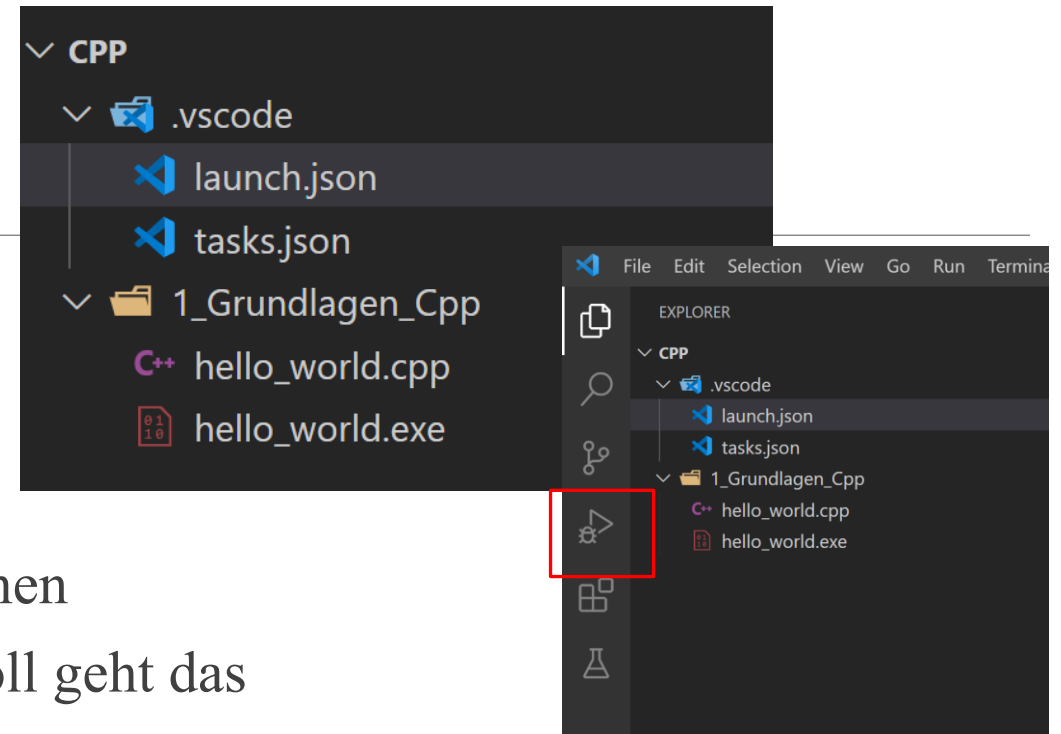
Debugging in VSCode

1. Mit VSCode den **übergeordneten** Ordner (Projekt-Ordner) öffnen indem Sie Dateien oder Unterordner speichern werden (z.B. ihren Ordner für die Cpp Vorlesung)
2. .cpp Datei öffnen (z.B. hello_world.cpp)
3. Dateipfad darf keine Leerzeichen enthalten!
4. Run → Start Debugging (F5)
5. Select Environment C++ GDB (GNU Debugger)
6. g++.exe Aktive Datei erstellen und debuggen



Debugging in VSCode

- Jetzt sollten Sie einen `.vscode` Ordner haben mit folgendem Inhalt:
- „`launch.json`“ definiert die Launch Befehle, die über das Debug Symbol links gestartet werden können
- Wenn der letzte Befehl nochmal gestartet werden soll geht das über F5.
- „`tasks.json`“ beinhaltet tasks die direkt auch von anderen Skripten in „`launch.json`“ aufgerufen werden können. Bei uns ist das das Kompilieren.
- Hier können Sie den Compiler und die Argumente des Compilers ändern.
- **Achtung:** Wenn noch eine ausführbare exe Datei vorhanden ist bricht das Debuggen bei neuen Fehlern nicht ab (wirft zwar einen Fehler in der Konsole) und nimmt danach die alte exe fürs Debuggen.



Kapitel 1: Einführung

1. „Hello C++ World!“ und Set-up
2. **Wiederholung C**

Aufgaben I: Funktionen, Kontrollstrukturen, IO

- 1) Schreiben, kompilieren und debuggen Sie das „Hello World!“ Programm.
- 2) Schreiben Sie mit Hilfe von `std::cin` ein Programm welches eine Zahl n über die Konsole bekommt und n^2 ausgibt.
- 3) Schreiben Sie eine Funktion welche $\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n$ für eine gegebenen Zahl n berechnet (wirklich über die Summe).
- 4) Schreiben Sie eine Funktion `int fib(int n)`, welche die n -te Fibonacci-Zahl F_n bestimme.
Hinweis: Die Fibonacci-Folge F_n ist definiert durch $F_0 = 0, F_1 = 1$ und $F_n = F_{n-2} + F_{n-1}$ für $n > 1$.
- 5) Schreiben Sie eine Funktion `bool prim(int n)`, die prüft ob n eine Primzahl ist (nur durch sich selbst und Eins teilbar).
Hinweis: `bool` ist ein elementarer Datentyp der genau zwei Werte (`true/false`) annehmen kann.

Aufgaben II: Funktionen, Kontrollstrukturen, IO

- 1) Implementieren Sie folgende Funktion und erklären Sie den Output:
- 2) Implementieren Sie folgende Funktion und erklären Sie den Output:

```
hello_world.cpp  test_integer.cpp X
1_Grundlagen_Cpp > C++ test_integer.cpp > ...
1  #include <iostream>
2
3  int main()
4  {
5      for (int n = 0; n >= 0; n += 100000)
6      {
7          std::cout << n << std::endl;
8      }
9  }
10
```

```
hello_world.cpp  C++ test_integer.cpp  C++ test_float.cpp X
_Grundlagen_Cpp > C++ test_float.cpp > ...
1  #include <iostream>
2
3  int main()
4  {
5      for (float x = 0.0; x != x + 1; x += 100)
6      {
7          std::cout << x << std::endl;
8      }
9  }
10
```

Kapitelübersicht

- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++
- 4 Die Standard Template Library (STL)
- 5 Clean Code
- 6 Test-driven Development

Kapitel 2: Objektorientierung

1. **Grundlagen**
2. Header Files
3. Vererbung
4. Polymorphismus

Exkurs: Git

Datenkapselung (C-Beispiel)

Datenkapselung (C++)

Was war nochmal prozedural?

- Trennung zwischen Funktionalität & Daten
- Zur Strukturierung stehen lediglich Funktionen (Prozeduren) zur Verfügung
- Variablen sind entweder lokal (innerhalb einer Funktion) oder, wenn übergreifend benötigt, auch global verfügbar
- Jede globale Variable kann überall bearbeitet werden
- Funktionen sind (sobald einmal deklariert) überall verfügbar
- Jede Funktion kann von überall mit jedem (vom Datentyp passenden) Parameter aufgerufen werden

Paradigmen der Objektorientierung

Prozedural

1. Orientiert sich an Funktionsweise von Rechnern (Ausführen von Anweisungen und Funktionen)
2. Strukturierung von Programmen nur über Funktionen und Structs
3. Trennung von Funktionalität und Daten
4. Daten sind nur lokal in Funktion oder Global verfügbar
5. Funktionen sind global verfügbar

Wartbarkeit schwierig!

VS

Objektorientiert

1. Orientiert sich an der „echten Welt“
2. Strukturierung über Funktionen, Structs und Objekte
3. Zusammenführen von Daten und Funktionalität
4. Kapselung: Daten-Verfügbarkeit definiert über Objekte
5. Kapselung: Funktions-Verfügbarkeit definiert über Objekte

4 Säulen der Objektorientierung

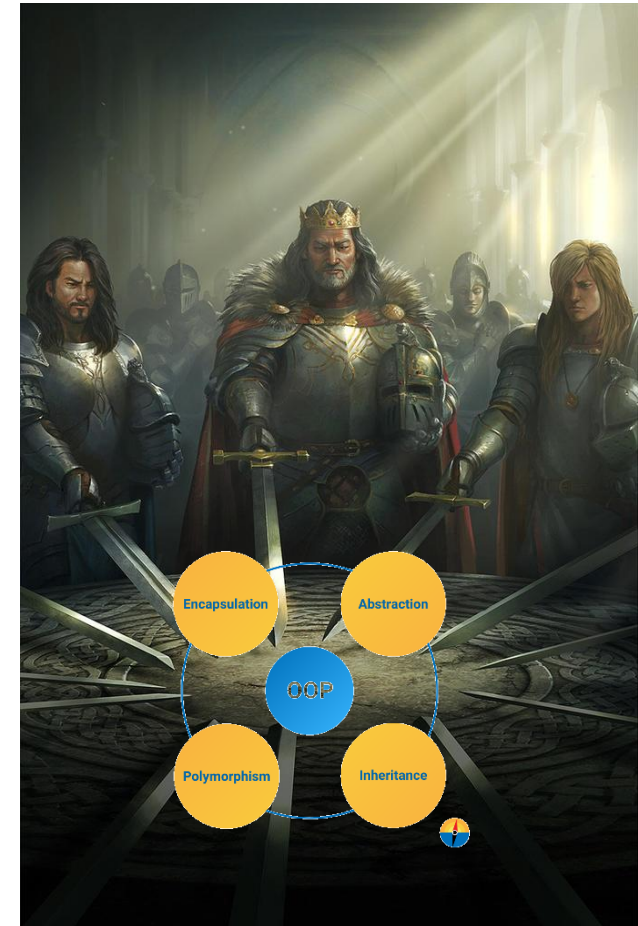
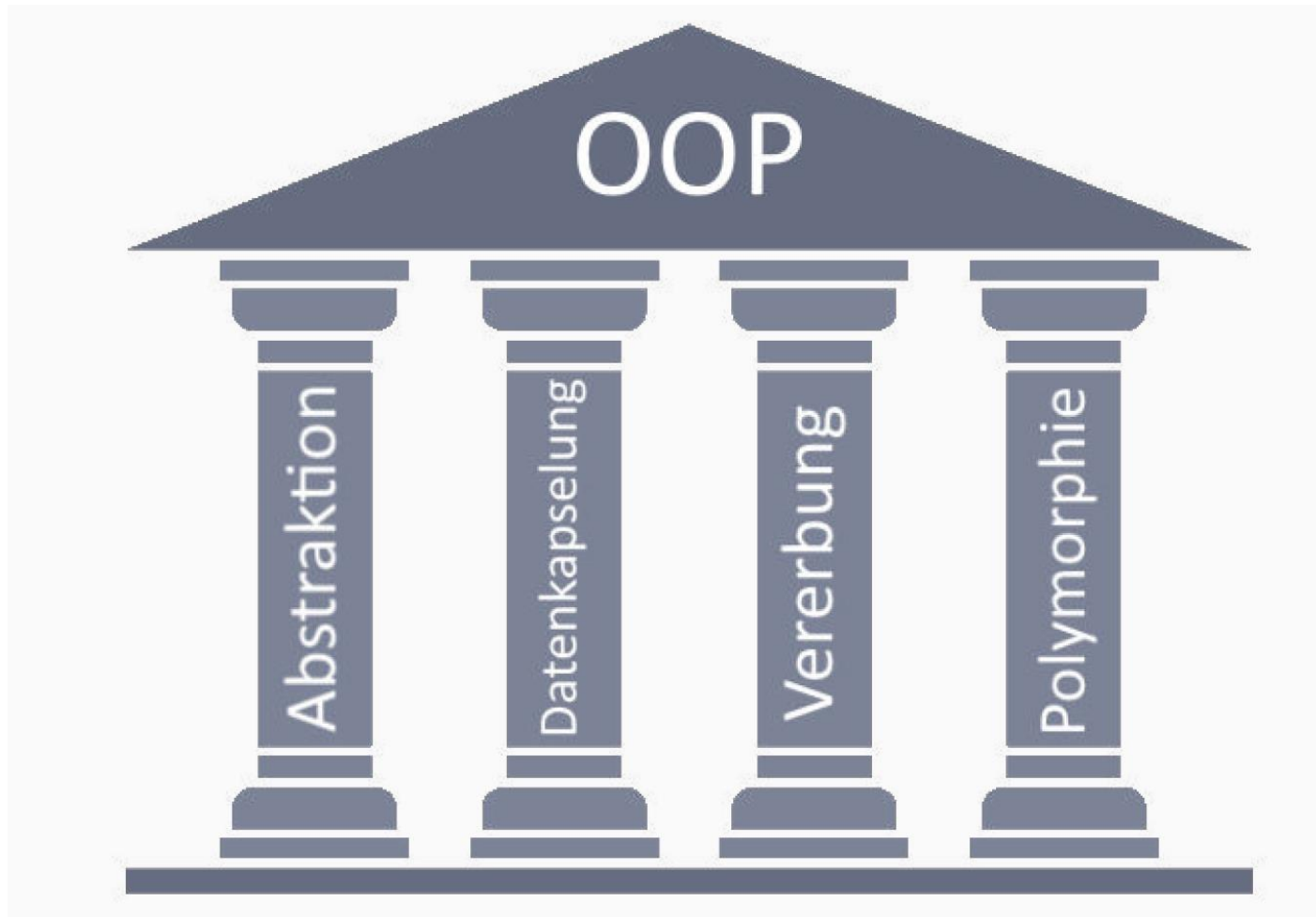
Einige Vorteile OOP

Bei richtiger Anwendung

- Gute Lesbarkeit
- Gute Erweiterbarkeit
- Intuitiver code

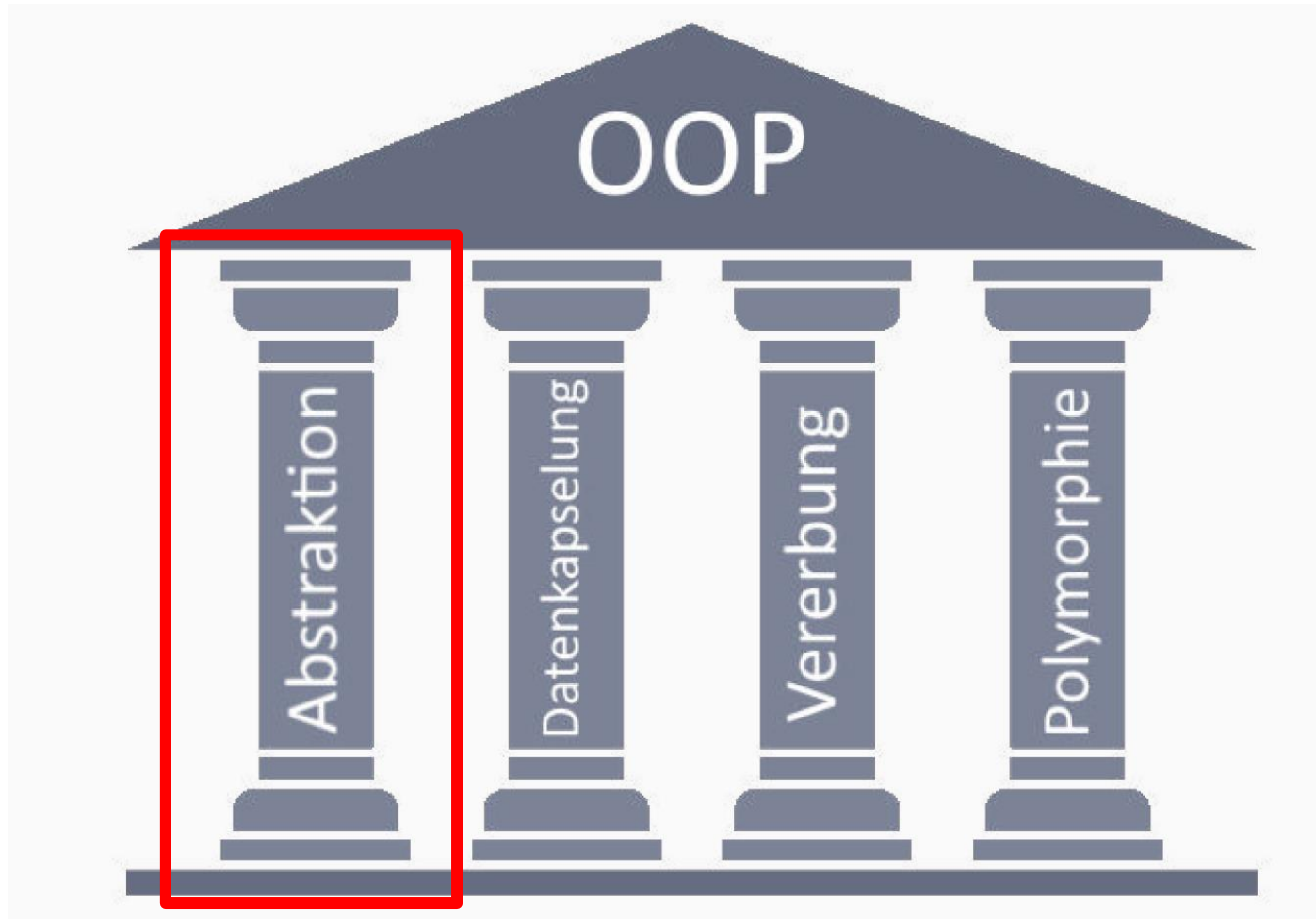


Die vier Säulen der objektorientierten Programmierung



<https://blog.devgenius.io/mastering-object-oriented-programming-and-solid-principles-a-comprehensive-guide-with-python-74acd2fb81c1>

Die vier Säulen der objektorientierten Programmierung



Objekte

Name	"Schiggy"
Nr. im Pokédex	7
Lebenspunkte	50
Level	1
Trainer	"Ash"



Was ist OBJEKTORIENTIERTE PROGRAMMIERUNG?
(Mit Pokémon erklärt) | [#Programmierung](#)
[Was ist OBJEKTORIENTIERTE PROGRAMMIERUNG?](#)
(Mit Pokémon erklärt) | [#Programmierung - YouTube](#)

Klassen und Objekte

Klassen: „Bauplan“

- Erweiterung von structs
- Enthält:
 - **Daten**
 - **Funktionalität**
- Kann Zugriff auf Werte verhindern
→ Kapselung!
- „Bauplan“ für das Erstellen von Objekten

Name
Nr. im Pokédex
Lebenspunkte
Level
Trainer

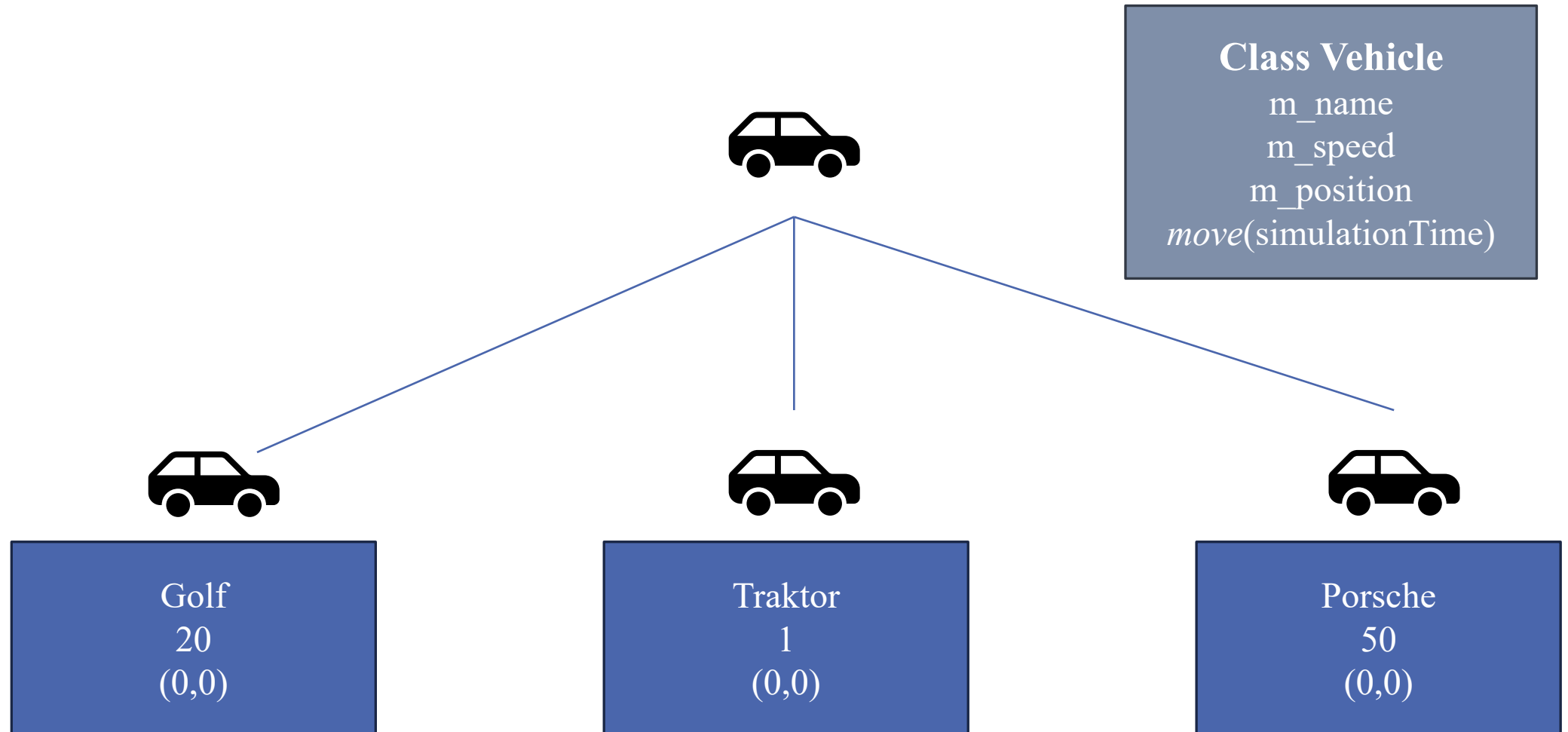
Objekte: „Instanz“

- Konkrete Instanz einer Klasse
- Enthält:
 - **Zustand (Daten)**
 - **Verhalten (Methoden)**
 - **Identität (z.B. über Speicherort)**

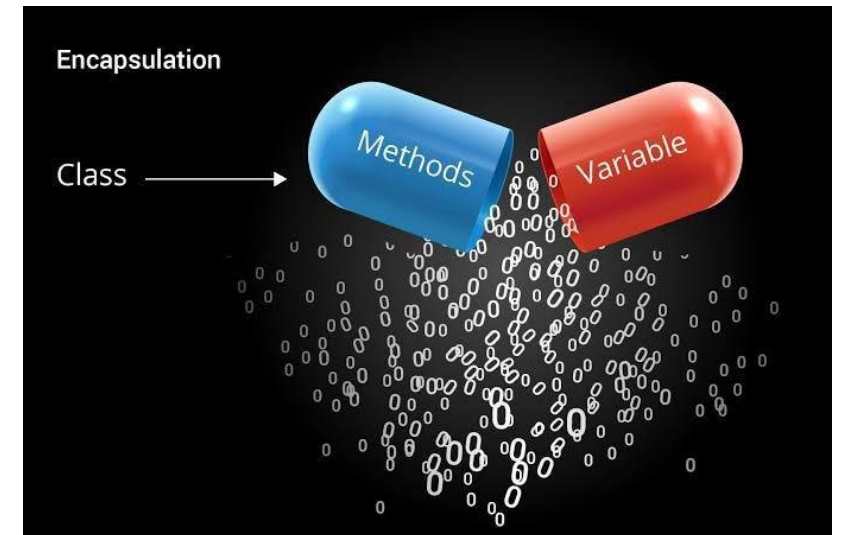
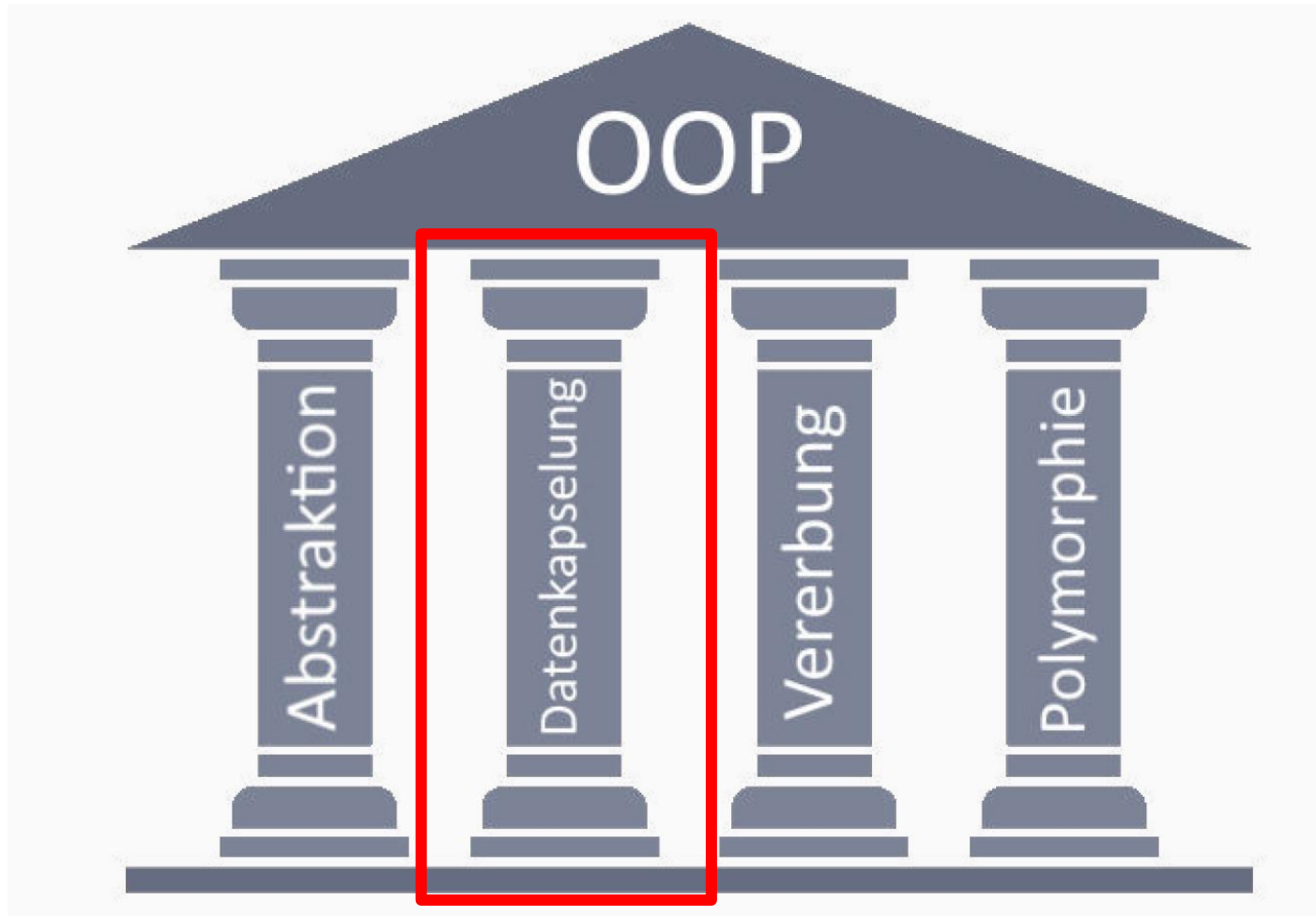
"Schiggy"
7
50
1
"Ash"



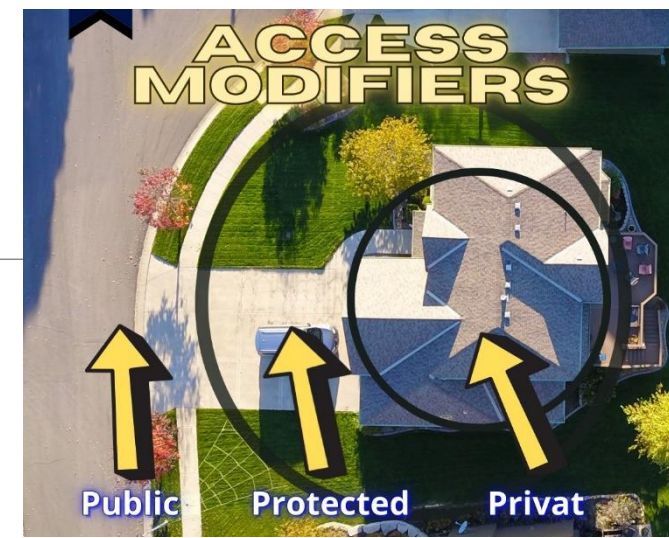
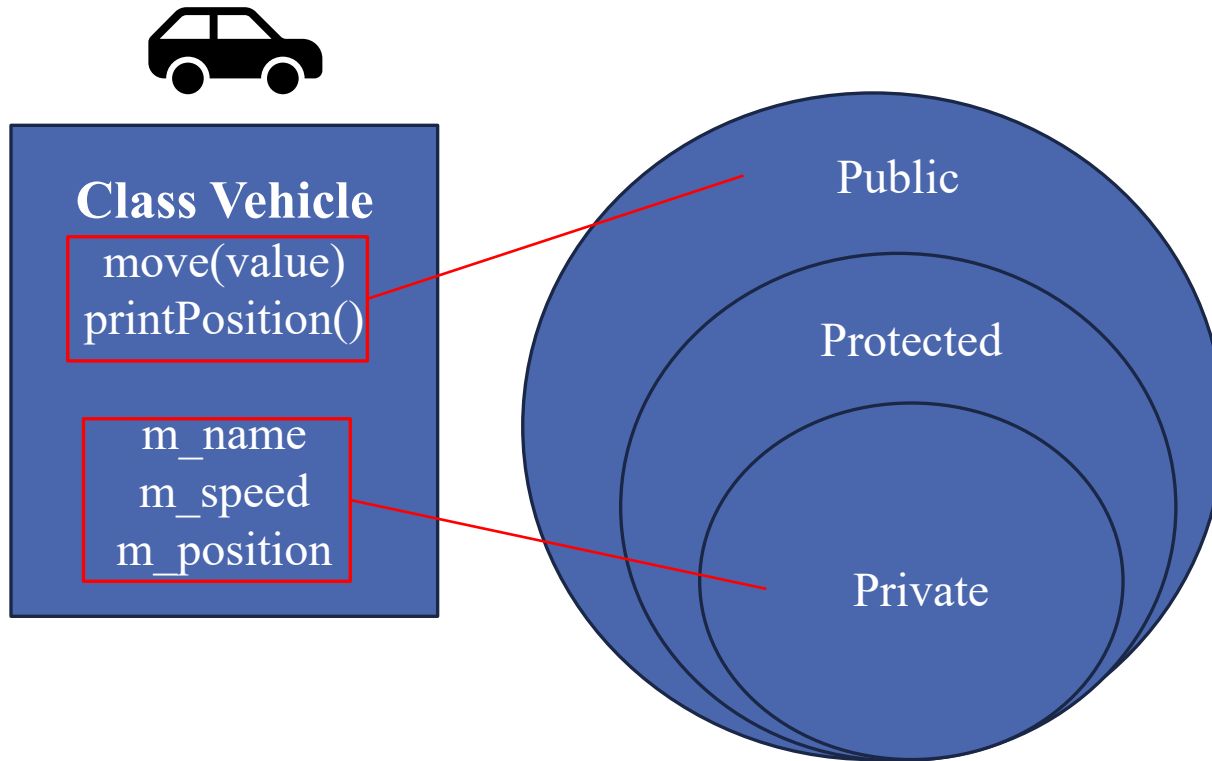
Klassen und Objekte



Die vier Säulen der objektorientierten Programmierung



Kapselung: Zugriffe



- **Public:** kann von außen aufgerufen werden, von einer Instanz können die Methoden etc. in diesem Bereich aufgerufen werden
- **Private:** kein Aufruf von außen möglich, Instanzen erreichen die Methoden und Membervariablen in diesem Bereich nicht (**auch Kinderklassen nicht**)
- **Protected:** Instanzen (reale Objekte) erreichen die Inhalte des Bereichs nicht, Kinderklassen erreichen diesen jedoch schon —> **Public für Kinder**

Konstrukturen & Initialisierungslisten & Überladen

Initialisierungslisten

```
1  #include <iostream>
2
3  class A
4  {
5  public:
6      A()
7      {
8          std::cout << "A";
9      }
10
11     A(int i)
12     {
13         std::cout << "X" << i;
14     }
15 };
16
17 class B
18 {
19 public:
20     B()
21     : m_a1()
22     , m_a2(2)
23     {
24         std::cout << "B";
25     }
26
27 private:
28     A m_a1;
29     A m_a2;
30 };
31
32 int main()
33 {
34     B();
35     return 0;
36 }
37
```

Frage: Was ist der Output dieses Programms?



Programmers. Everyday.



Überladen

```
2_Objektorientierung > C++ overload.cpp > main()
1  #include <iostream>
2
3  class Overload
4  {
5  public:
6      void foo()
7      {
8          std::cout << "noArgs ";
9      }
10
11     void foo(int)
12     {
13         std::cout << "int ";
14     }
15
16     void foo(int, int)
17     {
18         std::cout << "twoInts ";
19     }
20
21     void foo(double)
22     {
23         std::cout << "double ";
24     }
25 };
26
27 int main()
28 {
29     Overload obj;
30     obj.foo(4);
31     obj.foo(4.);
32     obj.foo();
33     obj.foo(3, 4);
34
35     return 0;
36 }
```

- Funktionen sind überladen wenn es mehr als eine Funktion mit dem gleichen Namen aber unterschiedlichen Parameter Listen gibt.
- Der Linker sucht nach der Funktion mit der am besten passenden Signatur.
- **Frage:** Welcher Output wird erzeugt?
- **Frage:** Welcher Output wird erzeugt wenn die letzte Funktion auskommentiert wird?

Aufgaben: Objektorientierung – Einführung (vehicle_example_V1)

- 1) Erstellen Sie eine Klasse Vehicle.
 - a) Dieses erhält beim Erstellen einen Namen, eine Position (nur x Wert als double), und einen durchschnittlichen Verbrauch (l/100km) und einen Tank mit 50 Litern Benzin.
 - b) Es soll zu einem Ort (x') fahren können wenn es genug Benzin für die Fahrt hat, sonst wird die Fahrt nicht angetreten.
 - c) Es soll betankt werden können.
 - d) Es soll ausgeben können wie viele Liter es seit Erstellung insgesamt verbraucht hat.

Testen Sie Ihre Klasse durch Aufruf in der `main` Funktion.
Testen Sie beim Entwickeln nach jeder neu hinzugefügten Eigenschaft!